BAB I
PENDAHULUAN

Bab ini menjelaskan dasar pengerjaan Tugas Akhir, mulai dari latar belakang dan rumusan masalah sampai tujuan, batasan masalah, metodologi, serta sistematika pembahasan.

I.1 Latar Belakang

Pemanfaatan *large language model* (LLM) memungkinkan pengembang membuat kode melalui instruksi bahasa alami. Kemampuan ini perlu disertai pemeriksaan keamanan karena kode yang menjalankan fungsi sesuai permintaan masih dapat memiliki kelemahan. CyberSecEval melaporkan bahwa model yang diuji memberikan saran kode rentan pada rata-rata sekitar 30% kasus dalam benchmark tersebut (Bhatt et al., 2023). Hasil ini menunjukkan bahwa kemampuan menghasilkan kode perlu diikuti pemeriksaan sebelum kode digunakan.

Pemeriksaan keamanan kode dapat memanfaatkan analisis statis dan penilaian LLM. *Static application security testing* (SAST) mencari pola kelemahan tanpa menjalankan program, tetapi dapat menghasilkan peringatan pada kode yang sebenarnya tidak rentan. LLM kemudian dapat membantu menilai peringatan berdasarkan konteks kode. QASecClaw menerapkan pembagian tugas tersebut dan melaporkan penurunan peringatan salah yang disertai kehilangan sebagian temuan benar (Ameen et al., 2026). Karena itu, penyaringan temuan perlu memperhatikan ketepatan peringatan sekaligus kelemahan yang terlewat.

Pendekatan multi-agen membagi pekerjaan pemeriksaan dan perbaikan kepada beberapa agen. AutoSafeCoder menghubungkan pembuatan kode, review, dan fuzzing melalui umpan balik berulang (Nunez et al., 2024). Multi-LLMSecCodeEval juga memadukan beberapa model dengan analisis statis pada tahap generasi, deteksi, serta perbaikan (Sabir et al., 2026). Pendekatan ini menyediakan dasar untuk menghubungkan pemeriksaan dengan perbaikan kode.

Dalam Tugas Akhir ini, perhatian diarahkan pada mekanisme yang menentukan apakah hasil pemeriksaan sudah cukup untuk menerima kode.

Mekanisme tersebut diperlukan ketika hasil alat dan agen berbeda atau pengujian belum dapat dijalankan. Kesepakatan reviewer juga masih perlu diperiksa bersama bukti yang tersedia. Refute-or-Promote melaporkan kasus ketika suatu pengujian membantah kandidat kelemahan yang didukung beberapa reviewer (Agarwal, 2026). Beyond QASecClaw turut menunjukkan pentingnya kesesuaian bukti dengan program yang diperiksa. Pada pengaturan Java yang diuji, penambahan konteks agen tidak memberikan manfaat yang konsisten dibandingkan kontrol deterministik (Al Mamun et al., 2026). Oleh karena itu, penggunaan beberapa agen perlu disertai aturan pengambilan keputusan dan pengujian manfaatnya.

Masukan yang dibaca reviewer juga dapat memengaruhi pemeriksaan. Penyerang dapat menambahkan komentar yang mengarahkan reviewer untuk mengabaikan kelemahan. Sanitizer dapat digunakan untuk menyiapkan masukan reviewer, tetapi pengaruhnya terhadap deteksi perlu diperhatikan. Thornton menemukan bahwa efek komentar adversarial tidak signifikan pada pengujian berpasangan, sedangkan penghapusan komentar dapat menurunkan deteksi pada model yang lebih lemah (Thornton, 2026). Ketahanan terhadap manipulasi karena itu perlu dinilai bersama kemampuan memeriksa kode tanpa serangan.

Tugas Akhir ini mengusulkan pengembangan *multi-agent security gate* dengan *Evidence-Based Arbiter* untuk memeriksa kode hasil LLM sebelum diteruskan kepada pengguna. Agen membantu menganalisis temuan, menyiapkan pengujian, dan memperbaiki kode. Arbiter non-LLM menggunakan bukti pemeriksaan dan aturan keamanan untuk menentukan apakah kode diterima (*pass*), diperbaiki (*revise*), atau ditahan untuk ditinjau manusia (*block*). Kode yang diperbaiki kembali menjalani pemeriksaan. Evaluasi diarahkan pada ketepatan keputusan, hasil perbaikan, dan ketahanan terhadap manipulasi reviewer. Verifikasi keamanan dalam penelitian ini berarti pemeriksaan terhadap persyaratan dan kategori kelemahan yang ditetapkan.

I.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, Tugas Akhir ini merumuskan pertanyaan sebagai berikut.

1. Bagaimana merancang dan mengimplementasikan multi-agent security gate dengan Evidence-Based Arbiter untuk menangani bukti pemeriksaan yang tidak lengkap atau bertentangan?
2. Bagaimana ketepatan keputusan dan hasil perbaikan gate dibandingkan dengan pemeriksaan menggunakan SAST saja serta SAST dengan satu reviewer LLM?
3. Bagaimana pengaruh sanitizer dan aturan arbiter terhadap kemampuan deteksi serta ketahanan gate ketika reviewer menerima komentar manipulatif?

I.3 Tujuan Tugas Akhir

Tugas Akhir ini bertujuan mengembangkan multi-agent security gate yang memeriksa dan memperbaiki kode hasil LLM dengan Evidence-Based Arbiter sebagai pengambil keputusan. Arbiter dirancang untuk menggunakan bukti pemeriksaan dan menangani hasil yang tidak lengkap atau bertentangan. Sistem dievaluasi dengan membandingkan ketepatan keputusan penerimaan serta hasil perbaikannya terhadap SAST saja dan SAST dengan satu reviewer. Evaluasi juga menilai pengaruh sanitizer dan aturan arbiter pada kode tanpa serangan maupun kode dengan komentar manipulatif, disertai pengukuran fungsi kode dan biaya pemeriksaan.

I.4 Batasan Masalah

Batasan masalah yang digunakan dalam Tugas Akhir ini adalah sebagai berikut.

1. Sistem memanfaatkan model LLM yang telah tersedia tanpa pelatihan atau fine-tuning model.
2. Pengujian menggunakan fungsi atau potongan kode Python dari tugas SecurityEval dan subset Python LLMSecEval (Siddiq & Santos, 2022; Tony et al., 2023). Fokus awal mencakup CWE-22, CWE-78, CWE-89, CWE-502, dan CWE-798. Contoh pola dan patch dalam knowledge store, jika digunakan, berasal dari data pengembangan yang terpisah dari data uji.

3. Analisis statis menggunakan Semgrep dan Bandit, sedangkan pengujian fungsi dan keamanan dijalankan dalam lingkungan terisolasi. Pemeriksaan wajib yang tidak dapat dijalankan dicatat sebagai tidak teruji dan menghalangi pass. Evaluasi dibatasi pada tugas yang dipilih, bukan aplikasi utuh atau lingkungan produksi.
4. Perbaikan otomatis dibatasi pada dua ronde setelah keluaran pertama. Keputusan block diarahkan untuk tinjauan manusia; pelaksanaan tinjauan tersebut berada di luar evaluasi sistem.
5. Manipulasi reviewer dibatasi pada komentar yang mempertahankan perintah dan perilaku program. Aturan arbiter, prompt sistem, hasil alat, dan knowledge store berada di luar akses penyerang. Sanitizer menyiapkan tampilan reviewer dengan mempertahankan kode asli untuk pemindaian dan eksekusi.

I.5 Metodologi

Pengembangan sistem dilakukan secara bertahap dengan pengujian pada data pengembangan sebagai dasar penyempurnaan rancangan. Konfigurasi dan kebijakan kemudian dibekukan sebelum evaluasi akhir. Tahapan pengerjaan Tugas Akhir adalah sebagai berikut.

1. **Studi literatur dan perancangan sistem.** Tahap ini mengkaji penelitian terkait dan menentukan kebutuhan gate. Hasilnya digunakan untuk menyusun arsitektur, pembagian peran agen, format bukti, dan aturan keputusan arbiter.
2. **Penyiapan data dan skenario pengujian.** Tahap ini memilih sampel, memeriksa label acuan serta duplikasi, dan memisahkan data pengembangan dari data uji. Skenario mencakup kode tanpa serangan, komentar manipulatif, bukti bertentangan, dan pemeriksaan yang gagal dijalankan.
3. **Implementasi sistem.** Tahap ini membangun alur pemeriksaan, sanitizer, pemanggilan alat, pengujian, arbiter, dan perbaikan kode. Bukti serta alasan keputusan dicatat untuk setiap versi kode, termasuk setelah patch.
4. **Pengujian dan analisis hasil.** Gate dibandingkan dengan generator tanpa gate, SAST saja, serta SAST dengan satu reviewer. Pengaruh sanitizer dan arbiter diuji melalui kondisi pembanding pada tugas yang sama. Analisis mencakup kode rentan yang lolos, penerimaan kode tidak rentan, hasil deteksi, keberhasilan

patch, fungsi kode, token, dan waktu pemrosesan. Hasil pada kode bersih serta kasus gagal turut dilaporkan.

I.6 Sistematika Pembahasan

Laporan Tugas Akhir ini terdiri dari lima bab dengan pembahasan sebagai berikut.

Bab I menjelaskan latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan sebagai dasar pengerjaan Tugas Akhir.

Bab II membahas keamanan kode hasil LLM, SAST, sistem multi-agen, perbaikan kode, dan ketahanan reviewer. Penelitian terkait dibandingkan untuk menjelaskan dasar rancangan Evidence-Based Arbiter.

Bab III menguraikan analisis kebutuhan serta perancangan solusi, termasuk arsitektur gate, peran agen, kebijakan arbiter, dan skenario evaluasi.

Bab IV menjelaskan implementasi sistem, konfigurasi eksperimen, hasil pengujian, serta analisis perbandingan sesuai rumusan masalah.

Bab V menyampaikan kesimpulan berdasarkan hasil evaluasi serta saran untuk pengembangan dan penelitian berikutnya.

DAFTAR PUSTAKA

Agarwal, A. (2026). *Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery*. https://arxiv.org/abs/2604.19049

Al Mamun, S. M. A., Islam, A., & Ameen, M. R. (2026). *Beyond QASecClaw: Agent Scaling, Verifier Feedback, and Evidence Reliability in Software Assurance*. https://arxiv.org/abs/2610.36947v1

Ameen, M. R., Alam, M. T. U., & Islam, A. (2026). *QASecClaw: A Multi-Agent LLM Approach for False Positive Reduction in Static Application Security Testing*. https://arxiv.org/abs/2605.01885

Bhatt, M., Chennabasappa, S., Nikolaidis, C., Wan, S., Evtimov, I., Gabi, D., Song, D., Ahmad, F., Aschermann, C., Fontana, L., Frolov, S., Giri, R. P., Kapil, D., Kozyrakis, Y., LeBlanc, D., Milazzo, J., Straumann, A., Synnaeve, G., Vontimitta, V., … Saxe, J. (2023). *Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models*. https://arxiv.org/abs/2312.04724v1

Nunez, A., Islam, N. T., Jha, S., & Rad, P. (2024). *AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing*. https://arxiv.org/abs/2409.10737v1

Sabir, B., Liu, S., Jang, S. I., Abuadbba, S., Gao, Y., Moore, K., Kim, S., Kim, H., & Nepal, S. (2026). *Does Teaming-Up LLMs Improve Secure Code Generation? A Comprehensive Evaluation with Multi-LLMSecCodeEval*. https://arxiv.org/abs/2603.22717

Siddiq, M. L., & Santos, J. C. S. (2022). SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 29–33. https://doi.org/10.1145/3549035.3561184

Thornton, S. (2026). *Can Adversarial Code Comments Fool AI Security Reviewers – Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis*. https://arxiv.org/abs/2602.16741

Tony, C., Mutas, M., Ferreyra, N. E. D., & Scandariato, R. (2023). *LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations*. https://arxiv.org/abs/2303.09384v1